STAT 479: HW5

Spring 2026

Submission Details

Upload your solution to Gradescope as a PDF document. Include all relevant figures within your explanations. Include all relevant code. You may use either R or python. You may review this assignment's source code here.

Evaluation Criteria

Technical Precision

- All quantities defined correctly.
- Claims are made precisely, no vagueness.
- Figures have labeled axes, legends, and descriptive captions.

Completeness:

- Each method choice is justified, not just described.
- Claims are grounded in readings, lectures, or original code.
- Logical connections between claims and evidence.
- Caveats are acknowledged where relevant.
- Justifications are given for each TRUE / FALSE question.

Clarity

- Appropriate formatting and demarcation into sections.
- Grammatically correct prose (not sentence fragments).
- Visualizations referenced in prose, not simply included without comment.
- No wordiness.

Code quality

- Only necessary outputs are shown, and manageable code size.
- Code is logically organized into functions or split into clearly commented sections.

- No unnecessary code or duplication.
- Logical variable names.

Problem 1 [Normalization]

a. The pseudocode below sketches Layer Normalization for a transformer layer. Is it correct? If not, correct it.

```
function layer_norm(X, beta, gamma):
    # Arguments:
    #   X is a (D, n_tokens) tensor
    #   beta is the LayerNorm bias parameter
    #   gamma is the LayerNorm scale parameter
    mu = row_means(X) # shape (D, 1)
    sigma_sq = row_variances(X) # shape (D, 1)
    X_norm = (X - mu) / sqrt(sigma)
    return gamma * X_norm + beta
```

b. TRUE FALSE In layer normalization, it is possible to choose β and γ so that the mean and variance of each sample across feature dimensions remains unchanged.

c. TRUE FALSE Batch normalization can be used for batch size $B = 1$.

Problem 2 [Vanishing and Exploding Gradients]

This problelm explores exploding and vanishing gradients in a 30 layer MLP $f_\theta(x)$. The model weights are randomly initialized using random normal variables with standard deviation `sigma_init`.

```python
import torch
import torch.nn as nn

def create_model(n_layers=30, D=64, sigma_init=0.3):
    """
    Initialize an MLP with random normal weights
    """
    layers = []
    for _ in range(n_layers):
        layers.extend([nn.Linear(D, D), nn.ReLU()])

    layers.append(nn.Linear(D, 1))
    for layer in layers:
```

```
        if isinstance(layer, nn.Linear):
            nn.init.normal_(layer.weight, std=sigma_init)
            nn.init.zeros_(layer.bias)
    return nn.Sequential(*layers)
```

a. Using `activations_norms` and `gradient_norms` (defined in the source qmd), run the code below and describe how $\|h^l\|_2$ and $\|\nabla_{W^l} f(x)\|_2$ change across layers l. Repeat with `sigma_init = 0.1` and comment on the implications for training deep models.

```
f = create_model()
x = torch.randn(1, 64)
act_norms = activation_norms(f, x)
grad_norms = gradient_norms(f, x)
```

b. Repeat (a) with `sigma_init` set to 0.1 instead of 0.3. What do both runs imply about training deep networks?

c. Modify the function `create_model` so that it includes an `nn.LayerNorm` layer after each ReLU layer. Rerun the analysis from (a) and comment on the differences. Refer to the definition of LayerNorm to explain this effect.

Problem 3 [Heteroskedastic Regression]

Beyond predictive performance, it is often necessary to measure the uncertainty associated with each prediction. This is especially useful in the *heteroskedastic* setting, where the residual error variance is not constant. Figure 1 plots training data with this property (link). A typical deep learning approach finds a nonlinear model f_θ that minimizes the mean-squared error loss

$$L_{\text{MSE}}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - f_\theta(x_i)\right)^2 \tag{1}$$

a. Instead of using an MSE loss, it is possible to optimize model parameters θ that instead minimizes the negative loglikelihood,

$$L_{\mathcal{N}}(\theta) = -\sum_{i=1}^{n} \log \mathcal{N}\left(y_i | \mu_\theta(x_i), \sigma_\theta^2(x)\right)$$

where both μ_θ and σ_θ must be learned during training. See also the code block below. Compare this loss with Equation 1: When are they equivalent, and what additional information does $L_{\mathcal{N}}(\theta)$ recover?

```
def nll_loss(y, mu, sigma):
    nll = torch.log(2 * torch.pi * sigma ** 2) + (y - mu) ** 2 / (sigma ** 2)
    return 0.5 * nll.mean()
```

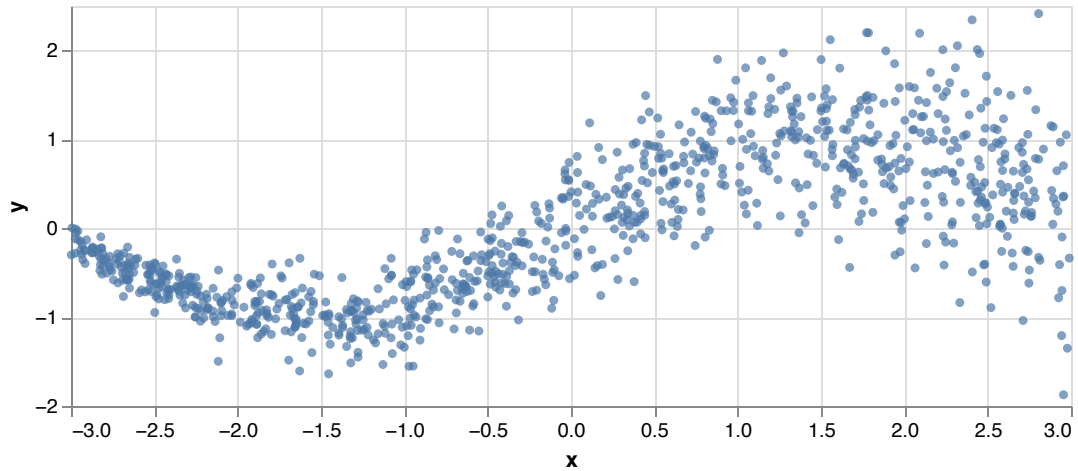

Figure 1

b. The block below defines an architecture that learns μ_θ and σ_θ in the equation above. Is it a perceptron, MLP, or transformer? Draw a picture of the network architecture and label the dimensionalities of the hidden layers and outputs.

```
class HeteroskedasticModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(1, 64), nn.ReLU(),
            nn.Linear(64, 64), nn.ReLU(),
            nn.Linear(64, 2)
        )

    def forward(self, x):
        out = self.net(x)
        mu = out[:, 0:1]
        sigma = torch.nn.functional.softplus(out[:, 1:2])
        return mu, sigma
```

c. The block below learns parameters for the model from (b) using the loss from (a). Visualize the predicted mean $\mu_\theta(x)$ and standard deviation $\sigma_\theta(x)$ over the test data (link).

```
x_train, y_train = to_tensors(train_df) # refer to .qmd source
model = HeteroskedasticModel()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

n_epochs = 2000
```

```
for epoch in range(n_epochs):
    optimizer.zero_grad()
    mu_hat, sigma_hat = model(x_train)
    loss = nll_loss(y_train, mu_hat, sigma_hat)
    loss.backward()
    optimizer.step()

print(f"Train loglikelihood: {-loss:.4f}")
```

```
Train loglikelihood: -0.3164
```

d. The block below computes the loglikelihood on the test split. Make a change to one of the following modeling decisions: The hidden state dimensionality, the number of layers, or the total number of epochs. After making the change, has the model generalization performance improved or worsened compared to the result in (c)? Justify your answer in terms of the gap between the training and test likelihood.

```
test_df = pd.read_csv("https://github.com/krisrs1128/stat479_notes/raw/refs/heads/master
x_test, y_test = to_tensors(test_df)

mu_test, sigma_test = model(x_test)
test_nll = nll_loss(y_test, mu_test, sigma_test).item()
print(f"Test loglikelihood: {-test_nll:.4f}")
```

```
Test loglikelihood: -0.4105
```

Problem 4 [Hotel Reviews]

A key property of deep networks is that they learn more and more abstract representations in deeper layers. In this exercise, you will visualize this phenomenon with a pretrained language model applied to a dataset of hotel reviews, both of which are loaded below.

Hint: You may want to initially develop your code on a subset of the original data, to speed up debugging.

```
from transformers import AutoTokenizer, AutoModel

reviews = pd.read_csv("https://github.com/krisrs1128/stat479_notes/raw/refs/heads/master/data
texts = reviews["Review"].tolist()

MODEL_ID = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
model = AutoModel.from_pretrained(MODEL_ID, output_hidden_states=True)
```

a. The model architecture can be viewed by typing `model` in a code block. Describe the role of `word_embeddings`. Where does the dimension 30,522 come from? What are the dimensions of the inputs and outputs for each transformer block?

b. The function `hidden_states` extracts activations for `texts` at a prespecified layer number. Use it to extract activations at layers 0 and 6.

```
def hidden_states(texts, layer_idx):
    x = tokenizer(texts, padding=True, truncation=True, max_length=128, return_tensors="pt")

    model.eval()
    with torch.no_grad():
        outputs = model(**x)

    hidden = outputs.hidden_states[layer_idx]
    return hidden.mean(dim=1).numpy() # average over tokens
```

c. For 2 - 5 reviews of your choice, find their $K = 10$ nearest neighbors with respect to Euclidean distance over representations at layers 0 and 6. What kinds of similarities does each layer capture?

d. Using layer 6's activations across the entire dataset, address: (i) Can they distinguish 1- vs. 5-star reviews? (ii) Among 5-star reviews, do reviews cluster into meaningful topics? You may use visualizations or additional models as you see fit.